

LABORATORY EXERCISE NO. 4

Working with Database and MVC in LavaLust

Course: Web Systems and Technologies 2

Framework: LavaLust PHP Framework

Laboratory Exercise No.: 4

Topic: Database Configuration, Model, View, Controller, and Data Retrieval

Estimated Time: 2–3 Hours

Mode: Individual Laboratory Activity

I. Learning Outcomes

At the end of this laboratory exercise, the student should be able to:

1. Create and configure a MySQL database for a LavaLust application.
2. Create a database table and populate it with sample records.
3. Configure the database connection in LavaLust.
4. Create a model that represents a database table.
5. Create a controller that communicates with the model.
6. Retrieve records from a database using the model's all() method.
7. Pass retrieved data from the controller to the view.
8. Display database records dynamically in an HTML table.
9. Apply the Model-View-Controller (MVC) architecture in a LavaLust application.

II. Learning Content

This laboratory exercise introduces the integration of a LavaLust application with a MySQL database.

III. Laboratory Scenario

You are developing a simple **User Management Module** using the LavaLust PHP Framework.

The application will retrieve user records from a MySQL database and display them in a web page.

The application must contain:

- A MySQL database named mydb
- A table named users
- A UsersModel
- A UsersController
- A user view
- A database configuration
- A route that accesses the controller
- A dynamic HTML table displaying all users

The application should follow the MVC architecture.

IV. Laboratory Procedures

Part A – Create the Database

Open your MySQL database management tool.

Create a database named:

mydb

You may execute:

CREATE DATABASE mydb;

Select the database:

USE mydb;

Part B – Create the users Table

Create a table named:

users

The table must contain at least the following fields:

Field	Data Type	Description
id	INT	Primary key
firstname	VARCHAR(100)	User's first name
lastname	VARCHAR(100)	User's last name
email	VARCHAR(150)	User's email address
username	VARCHAR(100)	User's username

Example SQL:

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    firstname VARCHAR(100) NOT NULL,  
    lastname VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL,  
    username VARCHAR(100) NOT NULL  
);
```

Note: You may add additional fields if required by your instructor.

Part C – Insert Sample Records

Insert at least **five (5) records** into the users table.

Example:

```
INSERT INTO users (firstname, lastname, email, username)  
VALUES  
('Juan', 'Dela Cruz', 'juan@example.com', 'juandelacruz'),  
('Maria', 'Santos', 'maria@example.com', 'mariasantos'),  
('Pedro', 'Garcia', 'pedro@example.com', 'pedrogarcia'),
```

```
('Ana', 'Reyes', 'ana@example.com', 'anareyes'),  
('Jose', 'Mendoza', 'jose@example.com', 'josemendoza');
```

Verify that the records were successfully inserted.

Execute:

```
SELECT * FROM users;
```

Part D – Configure the LavaLust Database

Locate the LavaLust database configuration file:

config/database.php

Configure the database connection using the database credentials of your local development environment.

The configuration must point to:

Database: mydb

Fill in the appropriate:

- Host
- Username
- Password
- Database name
- Port, if required

Example configuration values:

Host: localhost

Username: root

Password:

Database: mydb

Important: Use the actual database credentials configured on your computer. Do not simply copy the example credentials if your environment uses different settings.

Test the application to make sure LavaLust can establish a connection to mydb.

Part E – Create the Model

Create a model that represents the users table.

The model name must reflect the table name.

Create:

UsersModel

The model must be configured to use:

users

as its database table.

Conceptually:

UsersModel - users table

The model should contain the appropriate table configuration according to the LavaLust version being used in the laboratory.

Example concept:

```
class UsersModel extends Model
{
    protected $table = 'users';
}
```

Note: Follow the model structure and properties provided by your installed LavaLust version.

Part F – Create the Controller

Create a controller named:

UsersController

The controller name must reflect the users table.

The controller should load or access:

UsersModel

The controller will be responsible for retrieving all records from the users table.

Retrieve All Users

Inside the appropriate controller method, call the model's:

all()

method.

The basic logic should be:

UserController -> UserModel -> all() -> users table records

The retrieved records should be stored in a variable.

Conceptually:

```
$users = $this->UsersModel->all();
```

Use the appropriate LavaLust model syntax based on your installed version.

Part G – Pass the Data to the View

After retrieving the records, the controller must pass the data to the view.

The controller should perform the following operations:

1. Call UserModel.
2. Execute all().
3. Store the returned records.
4. Pass the records to the view.
5. Load the user view.

The intended flow is:

UserController

UsersModel::all()

\$users -> pass data -> View

Part H – Create the View

Create a view for displaying the users.

The view should contain an HTML table.

The table must display the following columns:

Column

ID

First Name

Last Name

Email

Username

The view must dynamically loop through the data received from the controller.

Conceptually:

<table>

Table Header

↓

Loop through users

↓

Display each user

</table>

Do **not** manually type each user into the HTML table.

The records must come from the database.

Part I – Create the Route

Create a route that connects the users URL to the UsersController.

For example:

/users

The route should call the appropriate method of:

UserController

V. Expected Output

When the student accesses:

/users

the browser should display a page similar to:

ID	First Name	Last Name	Email	Username
1	Juan	Dela Cruz	juan@example.com	juandelacruz
2	Maria	Santos	maria@example.com	mariasantos
3	Pedro	Garcia	pedro@example.com	pedrogarcia
4	Ana	Reyes	ana@example.com	anareyes
5	Jose	Mendoza	jose@example.com	josemendoza

The displayed records must originate from the MySQL users table.

VI. Functional Requirements

Your application must satisfy all of the following requirements:

Database

- Database named mydb exists.
- Table named users exists.
- Table contains the required fields.
- At least five records are stored in the table.

Configuration

- config/database.php is properly configured.
- Laravel successfully connects to mydb.

Model

- UserModel is created.
- The model is connected to the users table.

Controller

- UsersController is created.
- The controller uses UserModel.
- The all() method is used to retrieve records.

View

- User data is received from the controller.
- User records are displayed dynamically.
- An HTML table is used.

Routing

- A /users route is created.
- The route correctly calls UsersController.
- The page can be accessed through a web browser.

VII. Submission Requirements

Submit the following:

1. Database Screenshot

Show:

- mydb
- users table
- Table structure
- Inserted records

2. Database Configuration Screenshot

Show the relevant portion of:

`config/database.php`

Do not expose passwords in screenshots or submitted documentation.

3. Model Screenshot

Show:

`UsersModel`

4. Controller Screenshot

Show:

`UsersController`

including the use of:

`all()`

5. View Screenshot

Show the HTML/PHP/template code responsible for displaying the users.

6. Route Screenshot

Show the `/users` route.

7. Application Output

Submit a screenshot of the browser showing the dynamically generated users table.